

RC Car Self-Driving Code Walkthrough

Date: March 20, 2026

This document explains how the modular RC car controller works, from the smallest code elements up to the full control loop. It is written against the current code in:

- CODE/CONTROLLER/CURRENT/rc_car.py
- CODE/CONTROLLER/CURRENT/rc_car_app/config.py
- CODE/CONTROLLER/CURRENT/rc_car_app/hardware.py
- CODE/CONTROLLER/CURRENT/rc_car_app/lidar.py
- CODE/CONTROLLER/CURRENT/rc_car_app/vision.py
- CODE/CONTROLLER/CURRENT/rc_car_app/logging_utils.py
- CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py

1. The Big Picture

The current code is a self-driving RC car controller with four major jobs:

1. Read inputs from the human driver and sensors.
2. Decide what the car should do.
3. Send commands to the steering servo and drive motors.
4. Record telemetry to a CSV log.

The system is organized as a small package named `rc_car_app`. The top-level file `CODE/CONTROLLER/CURRENT/rc_car.py` is only a launcher. It imports `run()` from `rc_car_app.runtime` and starts the controller.

Reference: - `CODE/CONTROLLER/CURRENT/rc_car.py:1-6`

2. Smallest Building Blocks

2.1 Constants

Constants are fixed values that configure the system.

Examples in `config.py`:

- `ENABLE_HALL_SENSOR = True`
- `ENABLE_FRONT_ULTRASONIC = True`
- `ENABLE_RIGHT_ULTRASONIC = False`
- `ENABLE_WEBCAM_VISION = True`
- `AUTONOMOUS_CRUISE_PWM = 0.35`
- `AUTONOMOUS_TURN_PWM = 0.22`
- `AUTONOMOUS_TURN_STEER = 0.45`

- `FRONT_FAILSAFE_STOP_CM = 35.0`

These constants define the system behavior before the car ever starts moving. Feature flags enable or disable hardware subsystems. Tuning constants define how hard the car accelerates, how strongly it turns, and when it should stop.

Reference: - `CODE/CONTROLLER/CURRENT/rc_car_app/config.py:8-19`

2.2 Pin Definitions

GPIO pin constants map software names to real Raspberry Pi pins.

Examples:

- `STEERING_SERVO_PIN = 12`
- `HALL_SENSOR_GPIO_PIN = 24`
- `MOTOR_RIGHT_FWD_PIN = 19`
- `MOTOR_RIGHT_BWD_PIN = 20`
- `MOTOR_LEFT_FWD_PIN = 25`
- `MOTOR_LEFT_BWD_PIN = 13`
- `FRONT_ULTRASONIC_TRIG_PIN = 5`
- `FRONT_ULTRASONIC_ECHO_PIN = 6`

This means the program does not hard-code pin numbers all over the codebase. Everything reads from one place.

Reference: - `CODE/CONTROLLER/CURRENT/rc_car_app/config.py:21-31`

2.3 State Values

The function `create_state()` returns a dictionary of live runtime values.

Examples:

- `steer`
- `throttle`
- `brake`
- `gear_mode`
- `direction_arrow`
- `autonomous_mode`
- `lidar_front_dist`
- `camera_steering_bias`
- `camera_confidence`

This dictionary is the car's current operational state. It changes constantly while the system runs.

Reference: - `CODE/CONTROLLER/CURRENT/rc_car_app/config.py:99-123`

2.4 Metrics

The `Metrics` dataclass stores controller memory and derived quantities rather than immediate commands.

Examples:

- pulse_count
- smoothed_speed_mph
- max_speed_recall
- total_distance_cm
- pid_integral_error
- pid_previous_error
- pid_output
- stop_warning_start_time
- aeb_enabled
- aeb_triggered

This is not the same thing as state. State says what the car is doing now. Metrics say what the controller remembers from previous moments so that it can compute speed, PID output, and braking behavior over time.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/config.py:126-142

3. Configuration Layer

The file config.py is the root configuration layer for the whole program.

It defines:

- feature flags
- hardware pins
- motion tuning values
- speed conversion constants
- PID constants
- CSV log settings
- the initial state
- the Metrics dataclass

It is the one file you change first when you want to alter hardware usage or tune controller behavior.

Important design idea:

The code was intentionally configured for a stripped self-driving build. It keeps the drive train, steering, webcam, LiDAR, and optional safety sensors, but leaves out accessory outputs such as lights, horn, and matrix display.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/config.py:1-142

4. Hardware Layer

The file hardware.py is the hardware abstraction layer. It is responsible for creating Python objects that talk to the physical car.

4.1 Dummy Classes

The file starts by defining:

- DummyPWM
- DummyDigitalInput
- DummyDigitalOutput
- DummyServo

These exist so the program can still run even if GPIO initialization fails. That is useful for testing on a machine without full hardware attached.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/hardware.py:23–54

4.2 Hardware Initialization

The main class is Hardware.

Inside Hardware.__init__() the code creates:

- one steering servo
- four PWM outputs for the two drive motors
- a hall sensor input if enabled
- ultrasonic trigger and echo pins if enabled

If all of that works, gpio_initialized becomes True.

If any of it fails, the code falls back to dummy devices. This keeps the rest of the stack alive.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/hardware.py:57–95

4.3 Ultrasonic Reading

read_front_ultrasonic_distance() and read_right_ultrasonic_distance() are wrapper methods.

They do three things:

1. Check whether the feature is enabled.
2. Return simulated values if hardware is not initialized.
3. Otherwise call _read_hcsr04().

_read_hcsr04() performs the actual HC-SR04 measurement:

1. Pull trigger low.
2. Send a short trigger pulse.
3. Wait for the echo pin to go high.
4. Measure the pulse width.
5. Convert pulse width into distance using the speed of sound.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/hardware.py:97–140

4.4 Cleanup

`cleanup()` turns off the motors, closes sensor objects, centers the steering servo, and closes the servo.

This is important because the car should not keep driving or leave GPIO resources open after shutdown.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/hardware.py:142–170

5. LiDAR Layer

The file `lidar.py` handles LiDAR communication and spatial interpretation.

5.1 LiDAR Constants

This file defines:

- serial port `/dev/ttyUSB0`
- baud rate `230400`
- packet length `47`
- points per packet `12`
- `MAX_LIDAR_RANGE_M = 12.0`
- obstacle thresholds for caution and stop

These values define how raw LiDAR data is read and how close an obstacle must be to trigger warnings.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/lidar.py:8–14

5.2 LidarPoint

`LidarPoint` is the smallest LiDAR data structure. It stores:

- angle
- distance in millimeters
- confidence
- derived x and y coordinates
- validity

The method `calculate_cartesian()` converts polar data into Cartesian coordinates.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/lidar.py:17–30

5.3 LidarParser

`LidarParser` manages the serial connection and packet parsing.

Key methods:

- `connect()`: opens the serial port

- `disconnect()`: closes the port
- `read_data()`: reads raw bytes, splits them into packets, parses angles and distances, and stores points
- `get_latest_scan()`: returns the most recent full scan

The parser uses a byte buffer because serial data arrives as a stream, not as nicely separated messages.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/lidar.py:33-110

5.4 Direction Logic

`determine_turn_direction()` is the high-level LiDAR decision function.

It works like this:

1. Ignore invalid or low-confidence points.
2. Divide the scan into four sectors:
 - front
 - left
 - right
 - back
3. Find the minimum distance in each sector.
4. Decide:
 - STOP_WARNING if something is very close ahead
 - CAUTION_WARNING if something is moderately close ahead
 - ^ if front is open
 - < or > if a turn is better
 - v if reversing is safer
 - BLOCKED if no path is good

This function is the main geometric “free space” reasoner in the code.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/lidar.py:113-171

6. Vision Layer

The file `vision.py` handles the USB webcam.

6.1 Environment Path Setup

The first important thing `vision.py` does is add the local YOLO virtualenv site-packages path to `sys.path` if it exists. That allows this code to import `cv2` and `numpy` from your local bundled environment.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/vision.py:9-18

6.2 WebcamVisionProcessor

The main class is `WebcamVisionProcessor`.

It stores:

- device path
- camera handle
- thread state
- a lock
- current steering bias
- confidence
- timestamp of last processed frame

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/vision.py:21–32

6.3 Starting The Camera Thread

`start()` checks:

- feature flag enabled
- OpenCV available
- NumPy available

If the webcam can be opened, it sets:

- frame width 320
- frame height 240
- small capture buffer

Then it launches `_run()` on a background thread.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/vision.py:34–53

6.4 The Background Loop

`_run()` continuously:

1. reads a frame
2. skips failures
3. computes steering bias with `_estimate_path_bias()`
4. stores the result under a lock

This means vision processing runs asynchronously while the main control loop continues running.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/vision.py:67–77

6.5 Image Processing Logic

`_estimate_path_bias()` performs a lightweight classical-vision pipeline:

1. Get image height and width.
2. Crop the lower 45 percent of the image.
3. Convert to grayscale.
4. Blur with a Gaussian filter.
5. Detect edges using Canny.

6. Sum edge energy by column.
7. Compute the center of mass of that energy.
8. Normalize the left-right error into the range $[-1, 1]$.
9. Compute a confidence score from how much edge energy was detected.

The output is:

- a bias left or right
- a confidence score

This is not a full learned driving model. It is a simple vision feature extractor that estimates where visible structure lies in the lower camera image.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/vision.py:79–97

7. Logging Layer

The file `logging_utils.py` manages CSV output.

7.1 Logger Initialization

`init_csv_logger()`:

1. Opens the CSV file in append mode.
2. Creates a `csv.writer`.
3. Writes the header row if the file is empty.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/logging_utils.py:6–16

7.2 Writing Telemetry

`log_data_to_csv()`:

1. Computes elapsed time.
2. Computes average speed from total distance.
3. Computes time since last hall pulse.
4. Builds a row with state, metrics, sensor summaries, and system telemetry.
5. Writes the row and flushes the file.
6. Clears one-shot event flags.

It logs:

- autonomy state
- cruise state
- shift events
- steering, braking, throttle
- current speed and average speed
- LiDAR summary distances
- CPU and memory usage
- AEB status
- camera bias and confidence
- front ultrasonic distance

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/logging_utils.py:19–75

8. Runtime Layer

The file `runtime.py` is the top-level behavior controller. This is where the car actually “lives.”

8.1 Imports And Global Shutdown Flag

`runtime.py` imports:

- configuration constants
- hardware layer
- LiDAR layer
- logging utilities
- vision layer
- pygame for controller input
- `psutil` for telemetry

It also creates `shutdown_flag`, which is a thread-safe event used to stop the main loop cleanly.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:1–42

8.2 CPU Temperature

`get_cpu_temp()` runs `vcgencmd measure_temp` and parses the result. If anything fails, it returns `0.0`.

This is not a control function. It is just telemetry for logs.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:45–51

8.3 Speed Estimation

`calculate_speed()` converts hall sensor pulses into vehicle speed.

Detailed flow:

1. Measure time since the last speed update.
2. Count how many hall pulses happened since then.
3. Convert pulses into revolutions.
4. Convert revolutions into distance per second.
5. Convert to mph.
6. Smooth the value with exponential smoothing.
7. Update max speed.
8. Add the distance traveled during this loop to total distance.

This function is what turns raw wheel pulses into a stable speed estimate.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:54–83

8.4 Autonomous Fusion Logic

`apply_autonomous_controls()` is the main self-driving sensor fusion function.

Inputs:

- current state
- current metrics
- hardware object
- webcam vision processor

Steps:

1. Read front ultrasonic distance from hardware.
2. Save it into state for logging and downstream use.
3. Read camera steering bias and confidence from the webcam thread.
4. If the last camera frame is stale, zero out confidence.
5. Save camera values into state.
6. If the front ultrasonic sees something too close, stop immediately.
7. Read the LiDAR direction decision from `state["direction_arrow"]`.
8. If LiDAR says stop, blocked, or reverse, stop.
9. If LiDAR says forward or caution, drive forward and steer according to camera bias.
10. If LiDAR says left or right, turn that way while blending in some camera bias.

This is the core sensor fusion rule:

- ultrasonic has short-range emergency authority
- LiDAR has path and obstacle authority
- webcam refines steering inside the LiDAR-approved movement direction

Reference: - `CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:86–127`

8.5 Control Output Logic

`update_gpio()` converts high-level desired motion into actual servo and motor outputs.

It handles two major modes.

Autonomous Mode

If `state["autonomous_mode"]` is true:

1. Call `apply_autonomous_controls()`.
2. Set `state["throttle"]` and `state["brake"]` from those results.
3. Force cruise control off.
4. Reset the PID memory.

Reference: - `CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:135–144`

Manual Mode

If not in autonomous mode:

1. Clear vision and ultrasonic values used only by autonomy.
2. Handle gear logic:
 - P: stop and brake
 - N: stop without active brake
 - R: reverse throttle
 - D: forward behavior
3. If cruise control is active in Drive, run PID control.
4. Otherwise just use the human throttle input.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:145–191

Steering

The steering servo is set directly from state["steer"], clipped into [-1.0, 1.0].

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:193

AEB

Automatic emergency braking is only applied in manual mode.

Logic:

1. If LiDAR says STOP_WARNING, start a timer.
2. If the warning persists beyond AEB_ACTIVATION_DELAY_SEC, force braking.
3. Force desired PWM to zero.
4. Use a stronger brake rate.
5. Disable cruise control if it was active.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:195–215

Reverse Override

If the gear is R, the code overrides the AEB effects so the driver can reverse away from danger.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:217–220

PWM Smoothing

The code does not jump instantly to the desired motor command.

Instead:

- acceleration uses ACCEL_RATE
- active braking uses BRAKE_RATE or AEB_BRAKE_RATE
- passive slowdown uses COASTING_RATE

This makes the car less jerky.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:224–228

Motor Output Mapping

After the final PWM value is decided:

1. Clear all motor outputs.
2. If PWM is positive, drive forward outputs.
3. If PWM is negative, drive reverse outputs.

This is where abstract motion commands become GPIO PWM values.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:230-241

Speed Update

At the end of `update_gpio()` the code calls `calculate_speed()`.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:243

8.6 Main Application Startup

`run()` is the master function.

At startup it:

1. Prints a startup message.
2. Creates state.
3. Creates metrics.
4. Defines a `pulse_detected()` callback for the hall sensor.
5. Initializes Hardware.
6. Starts pygame.
7. Requires a joystick to be present.
8. Initializes the joystick object.
9. Starts LiDAR.
10. Starts webcam vision.
11. Starts CSV logging.
12. Creates the pygame clock.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:246-282

8.7 Main Loop

The main control loop runs while `shutdown_flag` is not set.

Each loop iteration does this:

1. Measure `dt`, the time since the last loop.
2. Process all pygame events.
3. Update state from joystick movement and button presses.
4. Read the latest LiDAR scan.
5. Convert that scan into direction and distance summaries.
6. Call `update_gpio()` to drive the car.
7. Log telemetry periodically.
8. Sleep enough to maintain roughly 60 Hz.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:284-401

8.8 Joystick Input Mapping

The controller logic is:

- left stick X axis controls steering in manual mode
- right trigger controls throttle
- left trigger controls brake
- button 4 toggles cruise control
- button 6 shifts down in P/R/N/D
- button 7 shifts up in P/R/N/D
- button 13 toggles autonomous mode
- button 14 toggles AEB
- button 15 quits
- D-pad up/down adjusts cruise target speed

Important safety behavior:

- pressing throttle cancels autonomous mode
- pressing brake cancels autonomous mode

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:290-361

8.9 LiDAR Update Inside Main Loop

During each frame:

1. `lidar_parser.read_data()` pulls in fresh serial bytes.
2. `get_latest_scan()` retrieves the current scan snapshot.
3. `determine_turn_direction()` computes movement intent and sector distances.
4. Those values are stored into state.

If LiDAR is unavailable, the code fills state with blank direction and max-range defaults.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:363-385

8.10 Logging Inside Main Loop

Every `LOG_INTERVAL_SEC`, the system logs one telemetry row using CPU, memory, temperature, state, and metrics.

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:389-399

8.11 Shutdown

On shutdown the code:

1. sets the shutdown flag
2. optionally logs one final row
3. disconnects LiDAR
4. stops webcam vision

5. cleans up hardware
6. closes the CSV file
7. quits pygame

Reference: - CODE/CONTROLLER/CURRENT/rc_car_app/runtime.py:403–424

9. How One Loop Iteration Works

To understand the whole system, it helps to imagine one single loop pass.

Step 1: The program enters the main loop in `run()`.

Step 2: It computes `dt`, the time since the previous loop.

Step 3: It checks joystick events. If you moved the stick or pressed a trigger, state changes.

Step 4: It reads LiDAR and updates:

- `direction_arrow`
- `lidar_front_dist`
- `lidar_left_dist`
- `lidar_right_dist`
- `lidar_back_dist`

Step 5: It calls `update_gpio()`.

Step 6: If autonomy is active, `update_gpio()` calls `apply_autonomous_controls()`.

Step 7: `apply_autonomous_controls()` reads:

- front ultrasonic
- camera steering bias
- LiDAR direction already stored in state

Step 8: It decides:

- stop
- go forward
- turn left
- turn right

Step 9: `update_gpio()` smooths the desired motor PWM and writes motor values.

Step 10: It writes the steering value to the servo.

Step 11: It updates speed from hall pulses.

Step 12: The main loop logs telemetry if enough time has passed.

Then the whole cycle repeats.

10. Sensor Roles

10.1 Hall Sensor

Purpose: - speed estimation

Used by: - `calculate_speed()`

Role: - tells the controller how fast the wheels are really moving

10.2 Front Ultrasonic

Purpose: - very short range emergency stop

Used by: - `apply_autonomous_controls()`

Role: - catches very close obstacles that should cause a hard stop

10.3 LiDAR

Purpose: - obstacle geometry and free-space direction

Used by: - `determine_turn_direction()`

Role: - decides what broad motion direction is safe

10.4 Webcam

Purpose: - soft steering refinement

Used by: - `WebcamVisionProcessor` - `apply_autonomous_controls()`

Role: - nudges steering left or right while LiDAR keeps authority over obstacle avoidance

11. Manual Mode Versus Autonomous Mode

Manual Mode

Manual mode is active when `state["autonomous_mode"]` is false.

In this mode:

- the joystick directly controls steering and throttle
- cruise control may be enabled
- AEB may intervene if LiDAR says stop

Autonomous Mode

Autonomous mode is active when `state["autonomous_mode"]` is true.

In this mode:

- joystick steering is ignored
- throttle/brake come from sensor fusion
- cruise control is disabled
- LiDAR and the webcam decide movement
- the front ultrasonic can force a stop

12. Why The Code Is Structured This Way

The split into modules makes the system easier to understand and safer to modify.

Benefits:

- `config.py` centralizes tuning
- `hardware.py` isolates GPIO details
- `lidar.py` isolates serial parsing and geometry
- `vision.py` isolates image processing
- `logging_utils.py` isolates telemetry
- `runtime.py` owns only the live behavior and orchestration

This is much easier to extend than one giant file.

13. Practical Limitations Of The Current Code

The current implementation is a strong v1 structure, but it still has limitations:

- webcam steering is classical vision, not a trained lane model
- LiDAR logic is sector-based, not full path planning
- there is no map or localization system
- there is no camera object detection in the active control loop
- no GPS integration is used
- no rear recovery camera is used

So this is a practical reactive self-driving base, not a full autonomy stack.

14. Summary

The entire system works as a layered stack:

1. `config.py` defines settings.
2. `hardware.py` talks to motors and sensors.
3. `lidar.py` interprets free space.
4. `vision.py` estimates a steering bias from the webcam.
5. `runtime.py` fuses all of that into steering and throttle.
6. `logging_utils.py` records what happened.

7. `rc_car.py` launches the system.

In plain language:

- the joystick and sensors tell the software what is happening
- the software decides what the safest useful motion is
- the servo and motor outputs make the car move
- the logger records the whole process for debugging